

19-ma'ruza. Statik metodlar. Metodlardan qiymat qaytarish. Metoddan chiqish.

Reja:

1. Statik metodlar. Parametrsiz metodlar
2. Metoddan chiqish
3. Koddan foydalanish mumkin bo'lmagan istisno

Metod parametrlarni belgilash orqali chaqiruvchi koddan ma'lumot olishi va qaytish turini ko'rsatib, chiqishni chaqiruvchi kodga qaytarishi mumkin. Qaytish turi void metodi uchun aniqlanishi mumkin, bu metod hech qanday qiymat qaytarmaydi. Metod, shuningdek qiymatni, *ref* va *out* parametrlari orqali ham chiqarilishi mumkin, ular haqida keying mavzularda to'xtalamiz.

Dasturlarni ishlab chiqishda kod miqdorini kamaytirish, uning o'qilishini yaxshilash va ushbu ko'rsatmalarni qayta ishlatishni osonlashtirish uchun ko'rsatmalarning ba'zi qatorlari takrorlanadi, ularni **metod**ga birlashtirish mumkin.

Agar o'zgaruvchilar ba'zi qiymatlarni saqlasa, u holda metodlar ma'lum harakatlarni bajaradigan ko'rsatmalar to'plamini o'z ichiga oladi. Asosan, metod - bu ba'zi harakatlarni bajaradigan nomlangan kod blokidir.

Parametrsiz metodlarning umumiy e'loni quyidagicha:

```
[modifikator] qiymat_turi Metod_nomi()
{
    // metod tanasi
}
```

Modifikatorlar va parametrlar ixtiyoriy.

Statik metodlar

“static” kalit so'zi bilan e'lon qilinadigan metodlar **statik metodlar** deb ataladi.

Statik metodlar barcha obyektlar uchun umumiy bo'lgan va ma'lum bir obyektga bog'liq bo'lmagan xatti-harakatlarni belgilaydi. Statik metodlarga kirish uchun sinf/struktura nomi ham ishlatiladi:

```
//Person sinfi kodi quyidagicha
using System;
namespace Funksiya
{
    class Person
    {
        public int Age { get; set; }
        static int P_yoshi = 65;
        public Person(int age)
        {
            Age = age;
        }
        public static void Status(Person J_yosh)
        {
            if (J_yosh.Age >= P_yoshi)
                Console.WriteLine("Nafaqaga chiqqan");
            else
                Console.WriteLine("Nafaqagacha: " + (P_yoshi - J_yosh.Age) + "yil bor");
        }
    }
}

//Program sinfi kodi quyidagicha
using System;
using Funksiya;
namespace Funksiya
```

```

{
    class Program
    {
        public static void Main(string[] args)
        {
            Person obyekt = new Person(55);
            Person.Status(obyekt);
            Console.ReadKey(true);
        }
    }
}

```

Bunday holda, Person sinfida statik Status() metodi aniqlanadi, u Person obyektini parametr sifatida qabul qiladi va uning *pensiya* holatini tekshiradi.

E'tibor bering, statik metodlar faqat sinfning statik a'zolariga kirishi mumkin. Statik metod ichidagi statik bo'lmagan metodlar, maydonlar, xossalarni nazarda tuta olmaymiz.

static kalit so'zini ko'rib chiqamiz. Ushbu static kalit so'z nafaqat metodlarga, balki o'zgaruvchilarga ham tegishli bo'ladi, chunki bu kalit so'z eng ko'p metodlar bilan ishlatiladi.

Sinf metodiga kirishda, ushbu sinfning obyektini yaratish kerak. Ammo dastur ishga tushganda Main() metodi tizim faqat qaysi metodni chaqirish kerakligini topadi, lekin uni sinf yaratmasdan chaqirish mumkinligi statik kalit so'zi orqali chaqirish mumkin.

Statik metodlarga sinf yaratmasdan kirish ham mumkin. Ammo darhol cheklov bo'ladi - statik metod faqat ushbu metod ichida e'lon qilingan o'zgaruvchilardan foydalanishi mumkin, lekin ular statik bo'lishi kerak. Bunday metod statik bo'lmagan tashqi ma'lumotlarga kira olmaydi, chunki obyekt yaratilmagan va agar obyekt yaratilgan bo'lsa, unda statik metod va ma'lumotlar unga tegishli emas, shuning uchun ishga tushirilmagan ma'lumotlardan foydalanish mumkin emas.

Statik metodlar va o'zgaruvchilar tizim tomonidan avtomatik ravishda yaratiladi va obyektga emas, sinfga biriktiriladi. Ya'ni loyihaga tegishli va yangi obyekt ishga tushirilganda statik o'zgaruvchilar uchun xotira ajratilmaydi. Sinfdan qancha obyekt yaratilishidan qat'iy nazar, har doim statik o'zgaruvchining faqat bitta ko'rinishi bo'ladi.

Shed sinfida *Obyekt* statik o'zgaruvchisini yarataylik, u dastlab nolga teng bo'ladi. Sinf konstruktorida o'zgaruvchining qiymati bittaga oshiriladi va public **GetObyekt()** metodi o'zgaruvchining qiymatini qaytaradi:

```

using System;
namespace Funksiya
{
    public class Shed
    {
        static int Obyekt = 0;
        public int GetObyekt()
        {
            return Obyekt;
        }
        public Shed()
        {
            Obyekt++;
        }
    }
}

```

Endi **Main()** metodida sinfning bir nechta obyektlarini yaratib, tekshirib ko'rishimiz mumkin:

```

using System;
namespace Funksiya

```

```

{
    class Program
    {
        public static void Main(string[] args)
        {
            Shed shed = new Shed();
            Console.WriteLine(shed.GetObyekt());
            Shed shed1 = new Shed();
            Console.WriteLine(shed1.GetObyekt());
            Console.ReadKey(true);
        }
    }
}

```

Sinfni ishga tushirib va birinchi obyekt yaratilgandan so'ng, *Obyekt* o'zgaruvchisining qiymati 1 ga teng bo'ladi va ikkinchi sinf obyekt yaratilgandan so'ng u o'zgaruvchining qiymati 2 ga teng. Agar yana boshqa obyekt yaratilsa, o'zgaruvchi yana bittaga ortadi.

Har bir obyekt o'zining statik bo'lmagan o'zgaruvchilariga ega va bir obyektida statik bo'lmagan maydonni o'zgartirish boshqa obyektlarga ta'sir qilmaydi. Statik maydon(o'zgaruvchi) har doim hamma uchun bir xil bo'ladi.

Statik o'zgaruvchilarni qanday ishga tushirish mumkin? Buni oddiy konstruktorda amalga oshirish mumkin, lekin keyin sinfning istalgan obyekt yaratilganda o'zgaruvchi qayta o'rnatiladi. Konstruktorda qandaydir tekshirishni amalga oshirish mumkin.

Eng yaxshi metod - oddiy konstruktor kabi sinf nomi bilan bir xil, lekin **static** kalit so'zi bilan e'lon qilingan statik konstruktordan foydalanish:

```

static Shed()
{
    Obyekt++;
}

```

Ushbu konstruktor quyidagi xususiyatlarga ega:

- sinfda yaratilgan obyektlar sonidan qat'iy nazar, faqat bir marta bajariladi;
- u parametrlarga ega bo'lishi mumkin emas, ya'ni uni qayta yuklash mumkin emas, bir nechta konstruktor yaratish mumkin bo'lmaydi va u faqat yuqorida aytib o'tilganidek bajariladi;
- konstruktorda kirish modifikatorlari yo'q, chunki u tashqaridan chaqirilmaydi, u sinfda birinchi obyekt yaratilganda yoki statik sinf a'zosiga birinchi marta kirishda avtomatik ravishda chaqiriladi.

Butun statik sinflarni ham yaratish mumkin. Agar sinf statik deb e'lon qilingan bo'lsa, u faqat statik o'zgaruvchilar va metodlarni o'z ichiga olishi mumkin:

```

using System;
namespace Funksiya
{
    static class MyStaticParams
    {
        public static void SomeMethod()
        {
            //Kod
        }
    }
}

```

Statik sinflar odatda, butun dastur uchun mavjud bo'lishi kerak bo'lgan, ba'zi yordamchi o'zgaruvchilar yoki metodlarni birlashtiradi. Boshqa dasturlash tillarida bunday o'zgaruvchilar

global deb nomlangan va dastur davomida ko‘rinib turgan. Sof obyektga yo‘naltirilgan dasturlash tilida sinfdan tashqarida metodlar va o‘zgaruvchilar bo‘lishi mumkin emas.

Sinfning statik a‘zolari obyektga emas, sinfga tegishli, shuning uchun ularga sinf nomi orqali murojaat qilish kerak. Statik bo‘lmagan ma’lumotlar obyektga nisbatan, sinfdan obyektga ishga tushirish va obyekt o‘zgaruvchisi orqali obyekt a‘zolariga kirish kerak. Sinfning statik a‘zosini chaqirish uchun quyidagilardan birini yozish kerak:

Sinf_nomi.O‘zgaruvchi

yoki

Sinf_nomi.Metod_nomi()

Masalan, yuqorida bayon qilgan statik metod shunday deyiladi:

MyStaticParams.SomeMethod();

Nuqtaning chap tomonida obyekt o‘zgaruvchisi emas, balki sinf nomi joylashgan.

Statik o‘zgaruvchilar haqida eslatma. Ularning qiymati birinchi murojaatda ishga tushiriladi. Bu shuni anglatadiki, agar dasturda 100 ta statik o‘zgaruvchi e‘lon qilingan bo‘lsa, bu ularning barchasi dastur boshida darhol ishga tushiriladi va ularning barchasi uchun xotira ajratiladi degani emas. Agar shunday bo‘lsa, dastur juda ko‘p vaqtni oladi va u juda ko‘p xotirani egallaydi. Aslida, xotira kerak bo‘lganda ajratiladi. Statik o‘zgaruvchiga birinchi marta kirishda uning uchun xotiradan joy ajratiladi va unga qiymat kiritiladi.

Garchi statik metodlar va o‘zgaruvchilar foydalanish nuqtayi nazaridan ancha tejamkor va ba‘zan qulay bo‘lsa-da, ulardan faqat haqiqatan ham kerak bo‘lganda foydalanish zarur.

Metoddan chiqish

Metoddan chiqish ikki holatda bo‘lishi mumkin. Birinchisi, yuqoridagi dasturdagi metodning tanasini yopuvchi figurali qavsga kelganda. Ikkinchisi, return operatori bajarilganda. return operatorining ikkita shakli mavjud: biri void tipidagi metodlar uchun, ya’ni qiymat qaytarmaydigan metodlar, ikkinchisi esa ma’lum qiymatlarni qaytaradigan metodlar uchun.

void turidagi metodni birdan tugatish uchun return operatorining quyidagi shaklini ishlatish kifoya.

```
return;
```

Ushbu operator bajarilganda, boshqaruv dasturni chaqirish qismiga qaytadi va metod tanasidagi qolgan kodlar o‘tkazib yuboriladi. Misol sifatida quyidagi metodni ko‘rib chiqamiz.

```
public void MyMeth()
{
    int i;
    for (i = 0; i < 10; i++)
    {
        if (i == 5) return; // 5-qadamda metoddan chiqiladi
        Console.WriteLine();
    }
}
```

Bu misolda for siklining 5 ta qadami bajariladi, chunki i o‘zgaruvchisi 5 qiymatga teng bo‘lganda metoddan chiqiladi.

Metodda bir nechta return operatorlariga ruxsat beriladi, ya’ni ikki yoki undan ortiq qaytarish variantlari mavjud bo‘lsa. Misol uchun:

```
public void MyMeth()
{
    //...
    if (done) return;
    // ...
    if (error) return;
}
```

Ushbu misolda metoddan chiqish ikki holatda sodir bo‘ladi: agar metodda done sharti bajarilsa yoki error sharti bajarilsa. Lekin ushbu holatda ehtiyotkorlik bilan foydalanish zarur. Metoddan juda ko‘p chiqish nuqtalari tufayli kodning strukturasi buzilishi mumkin.

Shunday qilib, void turidagi metoddan chiqish ikki holatda sodir bo‘lishi mumkin: yopilish figurali qavsiga yetganda yoki return operatori bajarilganda.

Qiymat qaytarish: void turiga ega bo‘lgan metodlar ko‘pincha dasturlashda qo‘llaniladi, ammo ko‘pchilik metodlar ma'lum bir qiymatni qaytaradi. Aslida, qiymat qaytarish metodning eng kerakli xossalardan biridir.

Qiymat qaytarish dasturlashda turli maqsadlarda qo‘llaniladi. Ba'zi hollarda, Math.Sqrt() metodida bo‘lgani kabi qiymat ba'zi hisob-kitoblarning natijasini o‘z ichiga olishi mumkin, metodning bajarilishini ko‘rsatishi mumkin. Qiymat qaytarishdan foydalanish C# tilida dasturlashning ajralmas qismi hisoblanadi.

Metoddan dasturning chaqirish qismiga qiymatni qaytarish uchun return operatorining quyidagi shaklidan foydalaniladi:

```
return qiymat;
```

bu yerda, qiymat – metodning qaytarish qiymati.

Masala: a va b sonlari ustida bitta arifmetik amal bajaruvchi metod yaratamiz va unda return operatori orqali qiymat qaytaramiz.

```
public static double Hisoblash(double a, double b, char op)
{
    double natija=0;
    switch (op)
    {
        case '-': {
            Console.WriteLine("- amali: \nnatija = ");
            natija=a-b;
            } break;
        case '*': {
            Console.WriteLine("* amali: \nnatija = ");
            natija=a*b;
            } break;
        case '/': {
            Console.WriteLine("/ amali: \nnatija = ");
            natija=a/b;
            } break;
        default: {
            Console.WriteLine("+ amali: \nnatija = ");
            natija=a+b;
            } break;
    }
    return natija;
}
```

Endi ushbu funksiyaga murojaat qilish uchun Main() metodida a va b haqiqiy sonlar va n1, n2, n3 operatsiyalar qiymatlarini kiritib quyidagi dastur matnini yozamiz:

```
public static void Main(string[] args)
{
    double a,b;
    char n;
    Console.WriteLine("a = ");
    a=double.Parse(Console.ReadLine());
    Console.WriteLine("b = ");
    b=double.Parse(Console.ReadLine());
}
```

```

        Console.WriteLine("Amal kiriting! op = ");
        n=char.Parse(Console.ReadLine());
        Console.WriteLine(Hisoblash(a,b,n));
        Console.ReadKey(true);
    }

```

Ushbu dastur bajarishi natijasida quyidagi natija olinadi:

a = 5

b = 9

*Amal kiriting! op = **

'' amali:*

natija = 45

Koddan foydalanish mumkin bo'lmagan istisno

Metodlarni yaratishda hech qanday vaziyatda kod bo'lagini bajarib bo'lmaydigan vaziyatlardan ehtiyot bo'lish kerak. Bunday kod mavjud emas deb ataladi va C# da yaroqsiz hisoblanadi. Agar kirish mumkin bo'lmagan kodni o'z ichiga olgan metod yaratilsa, kompilyator tegishli tarkibga ega bo'lgan ogohlantirish xabarini chiqaradi. Quyidagi dastur kodini ko'rib chiqamiz.

```

public void MyMeth()
{
    char a, b;
    // ...
    if (a == b)
    {
        Console.WriteLine("teng");
        return;
    }
    else
    {
        Console.WriteLine("teng emas");
        return;
    }
    Console.WriteLine("bu mavjud bo'lmagan kod ");
}

```

Ushbu misolda MyMeth() metodidan qaytarish har doim WriteLine() metodiga chaqirishni o'z ichiga olgan oxirgi e'lon bajarilishidan oldin sodir bo'ladi. Agar ushbu kodni kompilyatsiya qilinsa, ogohlantirish xabari chiqadi. Umuman olganda, natija bermaydigan kod dasturlash xatosi hisoblanadi.